

Problem Setup

```
f1 = x[1]^4 + (x[1] * x[2] - 1)^2
f2 = x[2]^2 * x[3]^2 + (x[3]^2 - 1)^2

f = f1 + f2
```

We are trying to verify Correlative Sparse relaxation for this problem is not accurate as the dense relaxation at degree 2.

cs_tssos_first (outer)

polys_info

To obtain the coefficient and support of all polynomials

nb denotes the number of binary variables i.e $x^2 = 1$

Inputs

```
pop = [x1^4 + x1^2x2^2 + x2^2x3^2 + x3^4 - 2x1x2 - 2x3^2 + 2]
x = [x1, x2, x3]
nb = 0
```

Outputs

```
n = 3
supp = [[1, 1, 1, 1], [1, 1, 2, 2], [2, 2, 3, 3], [3, 3, 3, 3], [1, 2], [3, 3],
[]]
coe = [[1, 1, 1, 1, -2, -2, 2]]
```

For example [1,1,1,1] represents monomial x_1^4

cs_tssos_first (inner)

Actual function for constructing the SDP and obtaining lower bound.

Input

```
supp = [[], [1, 1, 1, 1], [1, 1, 2, 2], [1, 2], [2, 2, 3, 3], [3, 3], [3, 3, 3, 3]]
coe = [[2, 1, 1, -2, 1, -2, 1]]
n = 3
d = 2
numeq = 0
nb = 0
CS = false
cliques = Any[]
basis = Any[]
hbasis = Any[]
minimize = false
TS = false
merge = false
md = 3
QUIET = false
solver = "COSMO"
```

```

tune = false
dualize = false
solve = true
solution = false
Gram = false
MomentOne = false
Mommat = false
tol = 0.0001
cosmo_setting = cosmo_para(1.0e-5, 1.0e-5, 10000, 0.0)
mosek_setting = mosek_para(1.0e-8, 1.0e-8, 1.0e-8, -1, 0)
normality = false
NormalSparse = false

```

Output

```

opt = 0.8498677416954203
sol = nothing
data = TSSOS.mcpop_data(3, 0, 0, 0, [[[]], [1, 1, 1, 1], [1, 1, 2, 0x0002],
[0x0001, 0x0002], [0x0002, 0x0002, 0x0003, 0x0003], [0x0003, 0x0003], [0x0003,
0x0003, 0x0003, 0x0003]]], Vector{Union{Number, AffExpr}}[[2, 1, 1, -2, 1, -2,
1]], Vector{Vector{Vector{UInt16}}}[[[]], [0x0001], [0x0002], [0x0003], [0x0001,
0x0001], [0x0001, 0x0002], [0x0002, 0x0002], [0x0001, 0x0003], [0x0002, 0x0003],
[0x0003, 0x0003]]], Vector{Vector{Vector{UInt16}}}[[]], Vector{UInt16}[[],
[0x0001], [0x0001, 0x0001], [0x0001, 0x0001, 0x0001], [0x0001, 0x0001, 0x0001,
0x0001], [0x0001, 0x0001, 0x0001, 0x0002], [0x0001, 0x0001, 0x0001, 0x0003],
[0x0001, 0x0001, 0x0002], [0x0001, 0x0001, 0x0002, 0x0002], [0x0001, 0x0001,
0x0002, 0x0003], [0x0001, 0x0001, 0x0003], [0x0001, 0x0001, 0x0003, 0x0003],
[0x0001, 0x0002], [0x0001, 0x0002, 0x0002], [0x0001, 0x0002, 0x0002, 0x0002],
[0x0001, 0x0002, 0x0002, 0x0003], [0x0001, 0x0002, 0x0003], [0x0001, 0x0002,
0x0003, 0x0003], [0x0001, 0x0003], [0x0001, 0x0003, 0x0003], [0x0001, 0x0003,
0x0003, 0x0003], [0x0002], [0x0002, 0x0002], [0x0002, 0x0002, 0x0002], [0x0002,
0x0002, 0x0002, 0x0002], [0x0002, 0x0002, 0x0002, 0x0003], [0x0002, 0x0002,
0x0003], [0x0002, 0x0002, 0x0003, 0x0003], [0x0002, 0x0003], [0x0002, 0x0003,
0x0003], [0x0002, 0x0003, 0x0003, 0x0003], [0x0003], [0x0003, 0x0003], [0x0003,
0x0003, 0x0003], [0x0003, 0x0003, 0x0003, 0x0003]], 1, [3], Vector{UInt16}
[[[0x0001, 0x0002, 0x0003]], Vector{UInt32}[[]], Vector{UInt32}[[]], UInt32[],
Vector{UInt16}[[0x0001]], Vector{Vector{UInt16}}[[[0x000a]]],
Vector{Vector{Vector{UInt16}}}[[[[0x0001, 0x0002, 0x0003, 0x0004, 0x0005, 0x0006,
0x0007, 0x0008, 0x0009, 0x000a]]]], Vector{Vector{UInt16}}[[]], nothing, nothing,
nothing, "COSMO", MathOptInterface.OPTIMAL, 0.0001, 1)

```

resort

Sorts the objective polynomial's coefficient and support based on the monomial order.

Input

```

supp[1] = [[1, 1, 1, 1], [1, 1, 2, 2], [2, 2, 3, 3], [3, 3, 3, 3], [1, 2], [3,
3], []]
coe[1] = [1, 1, 1, 1, -2, -2, 2]

```

Output

```

supp[1] = [[], [1, 1, 1, 1], [1, 1, 2, 2], [1, 2], [2, 2, 3, 3], [3, 3], [3, 3, 3, 3]]
coe[1] = [2, 1, 1, -2, 1, -2, 1]

```

get degree of constraints

```
dc = Int64[]
```

because this is unconstrained case

automatically generate cliques

Output

```

cliques = Vector{UInt16}[[0x0001, 0x0002, 0x0003]]
cql = 1
cliquesize = [3]

```

Input

```

n = 3
m = 0
numeq = 0
dc = Int64[]
supp = [[[]], [1, 1, 1, 1], [1, 1, 2, 2], [1, 2], [2, 2, 3, 3], [3, 3], [3, 3, 3, 3]]
d = 2
CS = false
minimize = false

```

if CS = true then record mc maximum clique size

assign_constraint

assign constraints according to cliques

Input

```

m = 0
numeq = 0
supp = Vector{Vector{UInt16}}[[[]], [1, 1, 1, 1], [1, 1, 2, 2], [1, 2], [2, 2, 3, 3], [3, 3], [3, 3, 3, 3]]
cliques = Vector{UInt16}[[1, 2, 3]]
cql = 1

```

Output

```

I = Vector{UInt32}[[[]]]
J = Vector{UInt32}[[[]]]
ncc = UInt32[]

```

set various degree limit in cliques

If `d` is "min" means different cliques can have different degree limit. It is most economic for Moment matrices.

```
d = 2
rlover = [2]
```

generate basis

Output

```
basis = [2]
hbasis = [[[]], [1], [2], [3], [1, 1], [1, 2], [2, 2], [1, 3], [2, 3], [3, 3]]
```

get_sbasis

What is the benefit of this `get_sbasis`?

Input

```
cliques[i] = [1,2,3]
rlover[i] = 2
basis[i][1] = [[
  [0x0001]
  [0x0002]
  [0x0003]
  [0x0001, 0x0001]
  [0x0001, 0x0002]
  [0x0002, 0x0002]
  [0x0001, 0x0003]
  [0x0002, 0x0003]
  [0x0003, 0x0003]]
```

get_blocks

input

```
NormalSparse = false
TS = false
I = Vector{UInt32}[]
J = Vector{UInt32}[]
supp = Vector{Vector{UInt16}}([[], [0x0001, 0x0001, 0x0001, 0x0001], [0x0001,
0x0001, 0x0002, 0x0002], [0x0001, 0x0002], [0x0002, 0x0002, 0x0003, 0x0003],
[0x0003, 0x0003], [0x0003, 0x0003, 0x0003, 0x0003]])
cliques = Vector{UInt16}([0x0001, 0x0002, 0x0003])
cql = 1
ksupp = nothing
basis = Vector{Vector{Vector{UInt16}}}([[], [0x0001], [0x0002], [0x0003],
[0x0001, 0x0001], [0x0001, 0x0002], [0x0002, 0x0002], [0x0001, 0x0003], [0x0002,
0x0003], [0x0003, 0x0003]])
hbasis = Vector{Vector{Vector{UInt16}}}[]
nb = 0
TS = false
merge = false
md = 3
```

```
n = 3
ss = nothing
```

output

```
blocks = [[[[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]]]]
eblocks = [[]] # ?
cl = [[1]] # clique length
blocksize = [[[10]]]
```

solvesdp

input

sadd

Adding exponents of two monomials. nb determines if binary variables are involved.

input

```
blocksize = [[[0x000a]]]
nb = 0
```